

CS-405: Deep Learning

Reinforcement Learning

Dr. Sajjad Hussain

SEECS, NUST

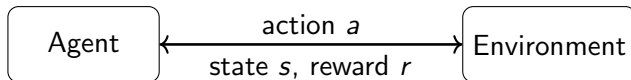
May 5th, 2026

Traditional ML vs Reinforcement Learning

Aspect	Supervised	Reinforcement
Data	(input, label) pairs	(state, action, reward)
Feedback	Immediate correct label	Delayed scalar reward
Goal	Predict label	Maximize cumulative reward
Interaction	No	Yes (agent $\leftrightarrow$ env)

Analogy: Training a dog vs. teaching with flashcards.

The RL Loop



Key elements:

- **Agent:** Learner/decision maker
- **Environment:** Everything outside the agent
- **Action (a):** What agent can do
- **State (s):** Environment's representation
- **Reward (r):** Immediate feedback (scalar)

Discount Factor γ

Discount factor defines how impatient the agent is to collect the future rewards. Future rewards are less certain.

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

γ close to 0	Future rewards become almost zero and are not required.
γ close to 1	Agent's reward goes down by very little on each time step.

A policy defines the agent's behavior:

$\pi(a \mid s)$ = Probability of taking action a in state s

Deterministic policy:

$$\pi(s) = a$$

Stochastic policy:

$$\pi(a_1 \mid s) = 0.7, \quad \pi(a_2 \mid s) = 0.3$$

State-Value Function $V^\pi(s)$

State Value function defines the discounted total reward the agent will get if it follows the given policy. i.e. How good is a state for the agent?

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

- Expected total discounted reward starting from state s
- Following policy π thereafter

Computing State-Value Function $V^\pi(s)$: Example Setup

- Grid: 3×3 , prize at $(2, 2)$: reward $+100$
- Step penalty: -1 for most cells
- **Special uphill cells:** $(1, 1)$ and $(2, 1)$ have penalty -2 (harder to traverse)
- Policy π (deterministic, given sequence):
 $(0, 0) \rightarrow (1, 0) \rightarrow (2, 0) \rightarrow (2, 1) \rightarrow (1, 1) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (1, 2) \rightarrow$
- Discount factor $\gamma = 1$ (no discount)

State-Value Function - Example

(0,0) -1	(0,1) -1	(0,2) -1
(1,0) -1	(1,1) -2	(1,2) -1
(2,0) -1	(2,1) -2	(2,2) +100

Step 1: Recall Definition of $V^\pi(s)$

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

For a **deterministic policy** with $\gamma = 1$:

$V^\pi(s)$ = sum of future rewards starting from s following π

We compute by **backwards induction**:

- 1 Start from terminal state $(2, 2)$
- 2 Move backwards along the policy path
- 3 Each state's value = immediate reward + value of next state

Step 2: Terminal State Value

At (2, 2): episode ends after receiving prize.

$$V^{\pi}(2, 2) = \text{reward at } (2, 2) = +100$$

Note: No future steps after terminal state.

Step 3: Compute $V^\pi(1, 2)$

Policy says: $(1, 2) \rightarrow (2, 2)$

Reward at $(1, 2)$: normal penalty = -1

$$V^\pi(1, 2) = \underbrace{(-1)}_{\text{reward at } (1,2)} + V^\pi(2, 2)$$

$$V^\pi(1, 2) = -1 + 100 = 99$$

Step 4: Compute $V^\pi(0, 2)$

Policy: $(0, 2) \rightarrow (1, 2)$

Reward at $(0, 2)$: normal penalty = -1

$$V^\pi(0, 2) = (-1) + V^\pi(1, 2)$$

$$V^\pi(0, 2) = -1 + 99 = 98$$

Step 5: Compute $V^\pi(0, 1)$

Policy: $(0, 1) \rightarrow (0, 2)$

Reward at $(0, 1)$: normal penalty = -1

$$V^\pi(0, 1) = (-1) + V^\pi(0, 2)$$

$$V^\pi(0, 1) = -1 + 98 = 97$$

Step 6: Compute $V^\pi(1, 1)$

Policy: $(1, 1) \rightarrow (0, 1)$

Crucial: $(1, 1)$ is an **uphill cell** $\rightarrow$ penalty is -2 (not -1)

$$V^\pi(1, 1) = \underbrace{(-2)}_{\text{special penalty}} + V^\pi(0, 1)$$

$$V^\pi(1, 1) = -2 + 97 = 95$$

Step 7: Compute $V^\pi(2, 1)$

Policy: $(2, 1) \rightarrow (1, 1)$

$(2, 1)$ is also an **uphill cell** $\rightarrow$ penalty -2

$$V^\pi(2, 1) = (-2) + V^\pi(1, 1)$$

$$V^\pi(2, 1) = -2 + 95 = 93$$

Step 8: Compute $V^\pi(2, 0)$

Policy: $(2, 0) \rightarrow (2, 1)$

Reward at $(2, 0)$: normal penalty = -1

$$V^\pi(2, 0) = (-1) + V^\pi(2, 1)$$

$$V^\pi(2, 0) = -1 + 93 = 92$$

Step 9: Compute $V^\pi(1, 0)$

Policy: $(1, 0) \rightarrow (2, 0)$

Reward at $(1, 0)$: normal penalty = -1

$$V^\pi(1, 0) = (-1) + V^\pi(2, 0)$$

$$V^\pi(1, 0) = -1 + 92 = 91$$

Step 10: Compute $V^\pi(0,0)$ — Start State

Policy: $(0,0) \rightarrow (1,0)$

Reward at $(0,0)$: normal penalty = -1

$$V^\pi(0,0) = (-1) + V^\pi(1,0)$$

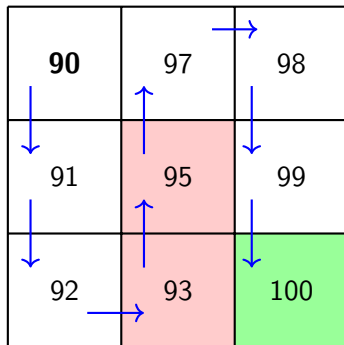
$$V^\pi(0,0) = -1 + 91 = 90$$

Complete State-Value Table for Policy π

State	Reward (penalty)	$V^\pi(s)$
(2, 2)	+100	100
(1, 2)	-1	99
(0, 2)	-1	98
(0, 1)	-1	97
(1, 1)	-2	95
(2, 1)	-2	93
(2, 0)	-1	92
(1, 0)	-1	91
(0, 0)	-1	90

Observation: Uphill cells (1, 1) and (2, 1) reduce value by an extra -1 compared to a normal path.

Visualizing $V^\pi(s)$ on the Grid



Takeaway: Value decreases as we move away from the prize, with extra penalty at uphill states.

Bellman Equation for State-Value Function

Our computed values (with $\gamma = 1$) satisfy:

$$V^\pi(s) = r(s) + V^\pi(s')$$

where:

- $r(s)$ = immediate reward in state s (-1 , -2 , or $+100$)
- s' = next state according to policy π (i.e., $s' = \pi(s)$)
- $V^\pi(s')$ = value of the next state (future rewards)

This generalizes to the **Bellman equation for a deterministic policy**:

$$V^\pi(s) = r(s, \pi(s)) + \gamma V^\pi(\pi(s))$$

where $\gamma \in [0, 1]$ is the discount factor (we used $\gamma = 1$).

State-Action Value Function $Q^\pi(s, a)$

We are more interested to know what action is most suitable in a given state. State-Action Value function is defined as the total reward received by the agent if it takes the action a in state s and keeps following the optimal policy thereafter.

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a]$$

Why Q is useful:

- Agent in state s can compare $Q(s, a)$ for all actions
- Pick the best: $\arg \max_a Q(s, a)$

Optimal policy from optimal Q:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Bellman Equation for Q (Deterministic Case)

The Q is recursive.

Take action a in s :

- 1 Get immediate reward r
- 2 Arrive at next state s'
- 3 From s' , repeat by continue following π

$$Q^\pi(s, a) = r(s, a) + \gamma Q^\pi(s', \pi(s'))$$

This self-consistency is the foundation of RL algorithms.

Computing $Q^\pi(s, a)$ for Deterministic Grid World

Recall:

$$Q^\pi(s, a) = r(s, a) + \gamma V^\pi(s')$$

where s' is the next state after taking action a in state s .

For our deterministic environment with $\gamma = 1$:

$$Q^\pi(s, a) = r(s, a) + V^\pi(\text{next state})$$

Policy π (deterministic):

$\pi(0, 0) = \text{Down}$, $\pi(1, 0) = \text{Down}$, $\pi(2, 0) = \text{Right}$, $\pi(2, 1) = \text{Up}$

$\pi(1, 1) = \text{Up}$, $\pi(0, 1) = \text{Right}$, $\pi(0, 2) = \text{Down}$, $\pi(1, 2) = \text{Down}$

Terminal (2, 2): no action needed.

Step 1: Known $V^\pi(s)$ Values (from earlier)

First, we must have the State Values available for the given policy.
From our earlier computations,

s	$V^\pi(s)$
(2, 2)	100
(1, 2)	99
(0, 2)	98
(0, 1)	97
(1, 1)	95
(2, 1)	93
(2, 0)	92
(1, 0)	91
(0, 0)	90

Step 2: Compute $Q^\pi(s, a)$ for All s, a

To compute the State-Value function for an action in a state, we look for immediate reward by performing action and the State-Value of the next state we land in. i.e.

Formula: $Q^\pi(s, a) = r(s) + \gamma V^\pi(s')$

e.g. To compute $Q^\pi(s = (0, 0), a = \textit{Right})$, the agent will land in $s' = (0, 1)$ if it goes *Right*. The immediate reward in current state $r(s = (0, 0)) = -1$, whereas the State-Value in new state $V^\pi(s' = (0, 1)) = 97$.

This gives, for discount factor $\gamma = 1$,

$$Q^\pi(s = (0, 0), a = \textit{Right}) = -1 + (1)97 = 96$$

Complete Table for Example

The following table is obtained for our example.

s	$r(s)$	$V^\pi(s)$	Left	Right	Up	Down	$\arg \max_a Q^\pi(s, a)$
(0,0)	-1	90	89	96	89	90	Right (96)
(1,0)	-1	91	90	94	89	91	Right (94)
(2,0)	-1	92	91	92	90	91	Right (92)
(2,1)	-2	93	90	98	93	91	Right (98)
(1,1)	-2	95	89	97	95	91	Right (97)
(0,1)	-1	97	89	97	96	94	Right (97)
(0,2)	-1	98	96	97	97	98	Down (98)
(1,2)	-1	99	94	98	97	99	Down (99)
(2,2)	+100	100	Terminal				

Table: Q-values with $\gamma = 1$. Bold = max in each row.

*Moving out of grid $\rightarrow$ stays in same cell $\rightarrow$ next state = same cell.

Deterministic is Too Limiting

In real environments:

- Robot's wheel slips
- Wind affects movement
- Sensor noise

We need **transition probabilities**:

$P(s' | s, a)$ = Probability of landing in s' after taking a in s

And possibly stochastic rewards:

$$R(s, a, s') \quad \text{or} \quad r(s, a)$$

Markov Decision Process

A Markov Decision Process is a tuple (S, A, P, R, γ) :

- S : finite set of states
- A : finite set of actions
- $P(s' | s, a)$: transition probability
- $R(s, a, s')$: reward (or $R(s, a)$)
- γ : discount factor $[0, 1]$

Markov Property:

$$P(S_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots) = P(S_{t+1} | S_t, A_t)$$

“Given present, future is independent of past.”

Stochastic MDP: Computing $Q^\pi(s, a)$

Setup:

Let's consider that for our same Grid World problem, we have probability of 0.7 that agent moves in the intended direction, and may slip and goes in other direction with probability of 0.1 in each of the three remaining directions. If any of the three directions goes out of bounds, then agent hits the wall and stays in the same cell.

- Intended action succeeds with $p = 0.7$
- Each other direction: 0.1 (total 1.0)
- Out of bounds $\rightarrow$ stay in same cell

Computing $Q^\pi((0,0), \text{Down})$ for Stochastic Environment

From (0,0) with action Down:

Down (intended)	0.7	$\rightarrow (1, 0), V = 91$
Left (slip)	0.1	$\rightarrow (0, 0), V = 90$
Right (slip)	0.1	$\rightarrow (0, 1), V = 97$
Up (slip)	0.1	$\rightarrow (0, 0), V = 90$

Formula:

$$Q^\pi(s, a) = r(s) + \sum_{s'} P(s' \mid s, a) V^\pi(s')$$

$$\begin{aligned} Q^\pi((0,0), \text{Down}) &= -1 + [0.7 \times 91 + 0.1 \times 90 + 0.1 \times 97 + 0.1 \times 90] \\ &= -1 + [63.7 + 9.0 + 9.7 + 9.0] = -1 + 91.4 = 90.4 \end{aligned}$$

Comparison: Deterministic vs. Stochastic Q^π for (0,0)

Action	Deterministic Q	Stochastic Q
Down	90.0	90.4
Right	96.0	94.0
Left	89.0	89.8
Up	89.0	89.8

Observations:

- Stochastic smoothing: extreme Q-values move toward mean
- Down becomes slightly better (slip into good state (0,1))
- Right becomes worse (risk of slipping into worse states)

Takeaway: In MDPs, Q-values depend on transition probabilities, not just next state.

Final Stochastic Q^π Table

The following Table is obtained for Stochastic environment.

s	$r(s)$	$V^\pi(s)$	Left	Right	Up	Down
(0,0)	-1	90	89.8	94.0	89.8	90.4
(1,0)	-1	91	90.4	93.6	89.8	91.0
(2,0)	-1	92	91.0	91.6	90.4	91.0
(2,1)	-2	93	91.2	96.0	93.0	91.8
(1,1)	-2	95	90.6	95.4	94.2	91.8
(0,1)	-1	97	91.0	95.8	95.2	94.0
(0,2)	-1	98	96.4	97.0	97.0	97.6
(1,2)	-1	99	95.2	97.6	97.0	98.2
(2,2)	+100	100	Terminal			

Table: Complete stochastic Q-table with $p = 0.7$ for intended action, $\gamma = 1$. Bold indicates $\max_a Q^\pi(s, a)$ in each state.

Disclaimer: Please double check as the values in the table may be incorrect.

Limitations of Q-tables

What's wrong with tables for real problems?

- **State explosion:**

- Atari game: 210×160 pixels, $256^{210 \times 160}$ states (impossible)
- Robot arm: continuous angles (infinite)

- **No generalization:** Never saw (2.1, 3.9) but need Q-value

Solution: Function approximation

$$Q(s, a) \approx f(s, a; \theta) \quad \text{with fewer parameters } \theta$$

From Table to Neural Network

Instead of a table, use a neural network:

$$Q(s, a; \theta) \approx Q^*(s, a)$$

Advantages:

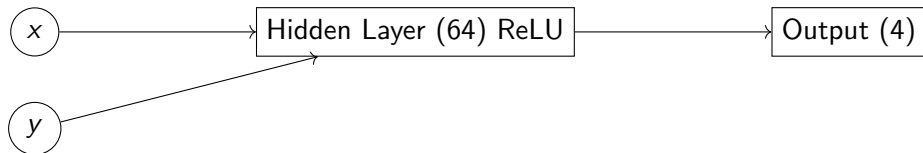
- Handles continuous states naturally
- Generalizes: similar states $\rightarrow$ similar Q-values
- Number of parameters $\ll$ number of states

Grid World Example with DQN

State representation: (x, y) coordinates $\in [0, 2] \times [0, 2]$

Network architecture:

- Input layer: 2 neurons (x, y)
- Hidden layer: 64 neurons with ReLU
- Output layer: 4 neurons (Up, Down, Left, Right Q-values)



After training: Network learns optimal Q-values without ever storing a full table.